

Corso di Laurea in Ingegneria e Scienze Informatiche

Progettazione di un configuratore di vendite

Tesi di laurea in:
BASI DI DATI

Relatore

Prof.sa Annalisa Franco

Candidato

Cavina Mattia

Sommario

To be written.

Indice

Elenco delle figure

Capitolo 1

Introduzione

Il progetto di tesi è volto alla progettazione ex-novo del programma di vendita dell'azienda Cepi S.p.a., azienda con una produzione di tipo **ETO!** (**ETO!**) specializzato nella progettazione di impianti di stoccaggio custom. La tipologia di produzione **ETO!** ha portato ad un profondo studio della situazione odierna, delle sue esigenze attuali e future tenendo aperte le possibilità di ampliamenti. L'analisi ha comportato un sostanziale lavoro di programmazione e raccolta dati per coprire nel modo più ampio i bisogni aziendali. Questa prima fase è stata condotta inizialmente attraverso la selezione di figure chiave scelte per mansione o/e ruolo, che potessero dare una visione dei bisogni del proprio o altrui reparto e successivamente sottoponendoli ad interviste. Sono state condotte 14 interviste, di cui 8 in modalità vis a vis, mentre le restanti 6 attraverso la dimostrazione dell'attuale applicativo utilizzato. Il risultato delle interviste condotte, ha fatto emergere attuali e nuove necessità architetture da affrontare come il cross-platform, l'utilizzo offline del database e la necessità di costruire offerte in modo più agile e strutturato. Questa prima fase ha inoltre sottolineato come il progetto non debba solo sostituire l'attuale sistema, ma debba inserirsi in un contesto di rinnovamento più ampio, coordinandosi con altri programmi aziendali e relativi database in ottica “**SRP!** (**SRP!**)” eleggendo dei master data che siano i soli responsabili della creazione di certi dati. Il **SRP!** ha plasmato notevolmente la costruzione del database ricreando, dove possibile, le strutture pre-esistenti da cui si necessitava recuperare i dati mediante reverse engineering. ?? L'intero sistema è stato pensato per ap-

poggiarsi all’infrastruttura esistente aziendale, come SQLServer[?], ma studiando delle logiche di flessibilità e ampliamento dinamico studiando accuratamente le relazioni tra le tabelle. Si è reso necessario cercare e scegliere inoltre un framework per la parte grafica che potesse soddisfare il deployment cross-platform. Questo requisito è emerso in modo chiaro dalle interviste, in quanto ci sono situazioni in cui il personale aziendale si trova a dover lavorare in mobilità, come le fiere di settore. La costruzione delle interfacce grafiche, è stata supportata dal sito “Figma”[?] che ha permesso una progettazione in ottica mobile first.?? La scelta accurata dei framework e le strutture dati nella fase iniziale, ha pesato molto in termini di tempo e progettazione, per garantire al richiedente un sistema che non rischiasse né il debito tecnico, né la preclusione a sviluppi futuri. La pianificazione dei primi test con gli utenti, dei rilasci delle demo e del metodo di progettazione sono state cruciali per dettare tempi e priorità degli item dovendo coprire in modo esaustivo almeno il core del progetto.

Capitolo 2

Situazione As-Is

2.1 Premessa

La situazione infrastrutturale aziendale è stata studiata dal sottoscritto nel corso del tirocinio formativo. La raccolta dei dati è stata svolta mediante interviste e studio dell'attuale sistema informatico. Nelle sezioni che seguono introdurrò brevemente l'As-Is.

2.2 Situazione infrastrutturale aziendale

L'infrastruttura informatica aziendale si appoggia su server fisici allocati internamente e gestiti mediante Active Directory. I server fisici sono, a loro volta virtualizzati per gestire separatamente le memorie in base alla finalità garantendo isolamento delle informazioni e permessi di accesso. Ogni reparto ha accesso a determinati server virtuali in cui gestisce in autonomia i propri contenuti. I database dei principali programmi sono gestiti all'interno di un server dedicato dove è installato un motore SQL Server.[?]. I principali database aziendali sono:

- **AdHoc[?]: l'ERP! (ERP!)** aziendale, programma che gestisce le attività aziendali come gli approvvigionamenti e la movimentazione dei prodotti, le vendite, le fatturazioni nonché i cicli produttivi.

- **Fusion[?]:** il **PLM!** (**PLM!**) aziendale, programma volto alla gestione dei prodotti meccanici, dalla loro creazione all'archiviazione dei disegni con relative distinte dei componenti. Altre funzioni applicate sui prodotti sono la gestione delle revisioni o versionamenti, ed infine la loro fine produzione.
- **BombConfine:** database ponte tra i database esterni e quello dell'**ERP!**. All'interno sono presenti tabelle di frontiera scritte da programmi esterni e che il gestionale usa per importare i dati.

La sicurezza aziendale è sostenuta da **NAT!** (**NAT!**) e Firewall che impediscono l'accesso dall'esterno se non tramite **VPN!** (**VPN!**). L'infrastruttura aziendale è visibile in figura ??.

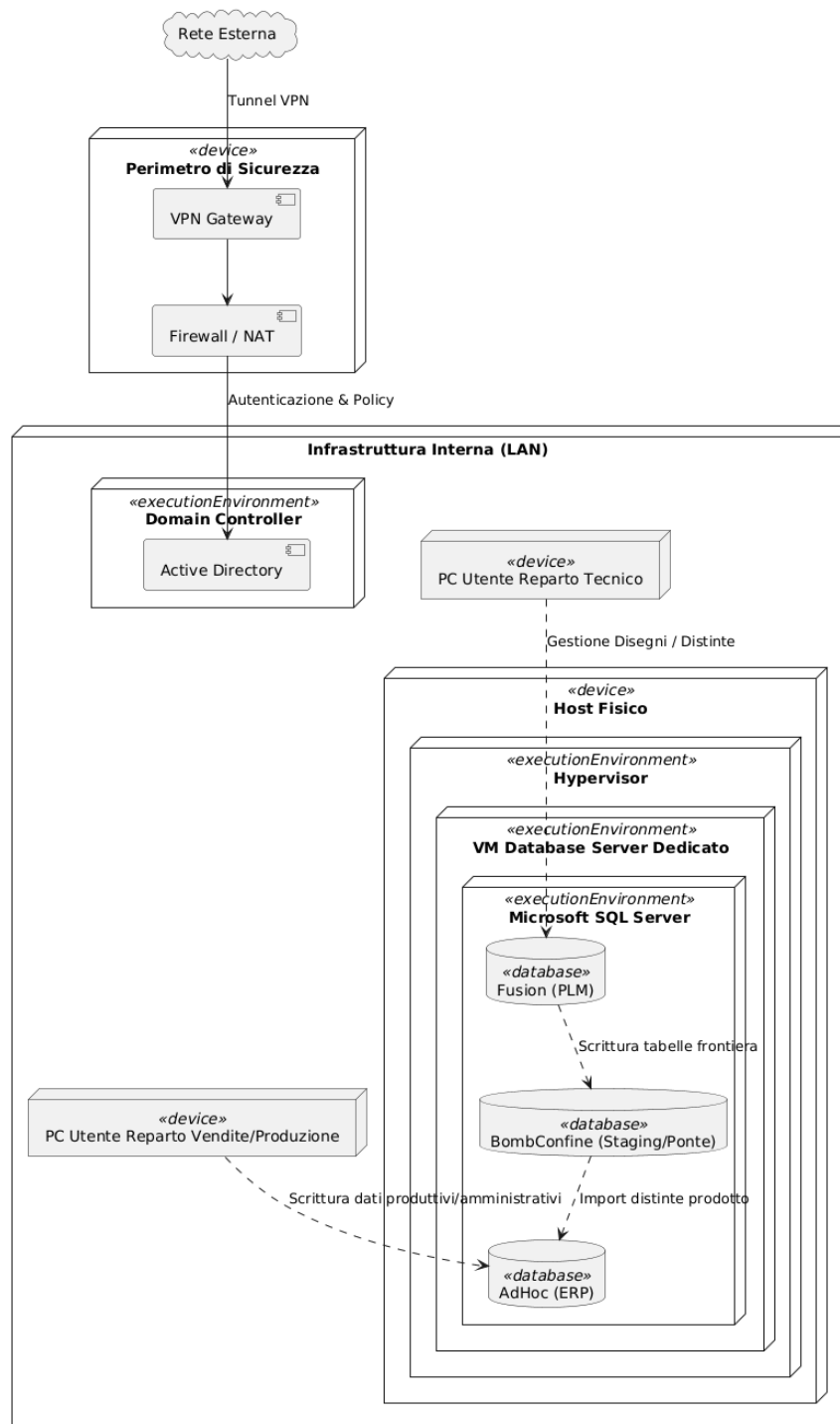


Figura 2.1: Schema infrastruttura aziendale.

2.3 Attuali software utilizzati

Nel capitolo precedente abbiamo fatto cenno ai database ed ai relativi programmi che gestiscono quanto concerne l'**ERP!** e il **PLM!**. Per quanto riguarda le vendite, punto focale dell'intervento richiesto, l'azienda utilizza attualmente un'applicazione sviluppata internamente. L'attuale sistema sviluppato in Access, programma della suite Microsoft Office, comprende al suo interno sia il frontend che il backend. Il software è stato implementato circa 30 anni fa ed ha subito numerosi aggiornamenti e modifiche nel tempo per adattarsi alle esigenze commerciali e all'evoluzione del mercato. L'applicazione consente di creare, modificare e gestire i contratti con i clienti, includendo funzionalità quali la gestione degli sconti, la personalizzazione delle offerte e la stampa dei documenti. Tuttavia, nonostante le funzionalità implementate, il software presenta significative criticità che ne limitano l'efficienza e l'usabilità, come confermato dalle interviste condotte con gli utenti. Tali criticità includono instabilità operativa, mancanza di standardizzazione, codifiche obsolete, difficoltà negli aggiornamenti di prezzo e assenza di integrazione con altri sistemi aziendali. Di conseguenza, è indispensabile sviluppare un nuovo programma che affronti tali problematiche e migliori l'esperienza degli utenti, preservando nel contempo le funzionalità essenziali del sistema attuale. Il programma Access è stato fondamentale in questi anni per l'utilizzo in zone offline. Questa funzionalità è stata favorita dalla natura del programma stesso, il quale racchiude in un solo file database e frontend, copiandolo in locale e sincronizzandolo una volta tornato in sede.

2.4 Organizzazione aziendale e flussi di lavoro

Il flusso lavorativo è risultato non essere un aspetto secondario nello sviluppo richiesto. Esso si deve interporre tra il reparto commerciale, ovvero addetti alle vendite, i reparti progettuali e produttivi come uffici tecnici, logistica ed amministrazione. L'attuale flusso di lavoro analizzato pone il reparto commerciale come punto di partenza dell'intero flusso che arriva fino al post-vendita. Il contratto, creato da tale reparto, dopo essere stato stipulato viene inviato al reparto amministrativo che ne importa i dati all'interno del gestionale. Simultaneamente esso

viene mandato al reparto project manager che lo revisiona tecnicamente in caso di errori progettuali. Dopo la revisione tecnica, il contratto viene spedito al reparto logistico che preventivamente verifica i dati di destinazione ed organizza le spedizioni sulla base del venduto. Il flusso di lavoro può essere visto in figura ??.

A causa del sistema obsoleto utilizzato ed il debito tecnico accumulato negli anni, l'attuale sistema è isolato rispetto ai programmi utilizzati dall'azienda. Quest'ultimo punto fa parte delle richieste imprescindibili da risolvere. L'integrazione nell'ecosistema informatico passa prima di tutto dall'unificazione dei codici, ovvero quelli a listino con i prodotti a gestionale, ma anche creando un flusso di notifiche tra i vari ambiti.

Capitolo 3

Raccolta dei requisiti

3.1 Scelta dei candidati

La raccolta requisiti si è svolta mediante la selezione di figure aziendali con distinti ruoli ed esperienza. Queste figure selezionate sono state poi sottoposte ad una intervista sia in modalità vis a vis, che in alcuni casi da remoto per favorire la registrazione del meeting. I candidati possono essere raggruppati in soci aziendali, utilizzatori, e non utilizzatori. All'interno degli ultimi due gruppi sono state selezionate, per far fronte ai basias dovuti all'abitudine, candidati sia con esperienza pluriennale che neoassunti. In totale sono state intervistate 14 persone così suddivise:

Utilizzatori abituali:

- **Agenti commerciali esterni:** 2 persone
 - 1 agente con esperienza limitata.
 - 1 agente con esperienza consolidata.
- **Commerciali interni:** 4 persone
 - 1 specializzato in nuovi impianti.
 - 1 specializzato in ricambi.

- 2 con competenze trasversali, sia per quanto concerne nuovi impianti che post-vendita.

Utilizzatori saltuari:

- **Soci aziendali con responsabilità:** 3 persone
 - 1 responsabile delle vendite.
 - 1 responsabile tecnico.
 - 1 responsabile amministrativo.
- **Direttore di produzione:** 1 persona

Non utilizzatori:

- **Impiegati tecnici:** 4 persone
 - 1 Project manager.
 - 1 Responsabile logistica.
 - 1 Progettista meccanico.
 - 1 Progettista elettrico.

L'ordine delle interviste svolte non è stato lineare, ma si è optato di partire dalla proprietà, per capire la direzione ed aspettative del progetto. Le restanti interviste hanno seguito un ordine logico, ma al contempo sparso, alternando utilizzatori e non utilizzatori, con l'obiettivo di non saturare ogni singolo reparto. Questa modalità, ha consentito di fare da guida per le interviste successive.

3.2 Modalità di svolgimento interviste

Le interviste, come esplicito in precedenza, si sono svolte singolarmente sia in presenza che da remoto. Con il permesso degli interlocutori sono state registrate, anche solo l'audio, per poi essere rielaborate e trascritte con supporto di **IA!** (**IA!**) generativa. La scelta di utilizzare la modalità fisica o da remoto è stata

dettata dalla necessità. Per gli utilizzatori abituali si è reso necessario registrare visivamente come utilizzano l'attuale applicativo. Tramite questa modalità si è potuto raccogliere i flussi di utilizzo e le principali funzioni presenti da mantenere. Nei restanti incontri si è preferito un approccio in presenza con l'interlocutore, facilitato la raccolta di requisiti e consigli sul progetto.

Le prime interviste sono state supportate da una scaletta di domande, preparate sulla base dell' AS-IS ?? che hanno guidato l'incontro. Con il progredire delle interviste, è stato adottato un approccio più flessibile che ha affinato i risultati delle interviste in previsione delle successive. Questa modalità ha consentito di ricavare i requisiti fondamentali ordinati per importanza.

3.3 Risultati interviste condotte

3.3.1 Rispetto all'attuale sistema

Funzionalità da mantenere

Dalle interviste condotte, è emersa l'importanza di mantenere una serie di funzionalità, quali:

- Suddivisione del contratto in capitoli che raggruppano porzioni di esso.
- Gestione delle scontistiche a cascata, ovvero la possibilità di inserirle rispetto alla singola voce contrattuale, nel capitolo, o dell'offerta stessa.
- Gestione di voci o capitoli opzionali, che non concorrono al prezzo offerto, ma ad uno opzionale.
- I campi del database che attualmente concorrono alla creazione dei report, sono da mantenere.
- Creazione reportistica formato PDF del contratto digitale. Da gestire, come ora, il multilingua del report stesso.
- Esportazione all'**ERP!** delle voci contrattuali presenti nell'ordine.

Criticità emerse

Mediante le dimostrazioni di utilizzo si è potuto raccogliere le criticità da attenzionare, come:

- Blocchi frequenti in condizione di concorrenza. Se più utenti utilizzano in contemporanea il file si generano deadlock.
- Non è presente una gestione centralizzata dei listini prodotto. Attualmente si gestisce esternamente tramite file Excel.
- Mancanza di linee guida nella compilazione delle offerte. Si è avanzata l'ipotesi di un configuratore di prodotto che aiuti l'utente.
- Mancanza di gestione dei valori duplicati e controlli sulle foreign key porta a creare clienti e prodotti duplicati.

3.3.2 Richieste avanzate

Gestione ruoli e permessi

La scelta di coprire quanti più reparti possibili, ha fatto emergere la mancanza di gestione dei ruoli. Si è richiesto quindi di attribuire ad ogni utente un ruolo, a sua volta ognuno di essi avrà viste sul database e permessi dedicati. I ruoli individuati sono:

- Admin con visione sull'intero sistema, con possibilità esclusiva di modifica ad alcune tabelle fondamentali.
- Commerciali interni che possono gestire totalmente i clienti e contratti.
- Commerciali esterni all'azienda, esempio agenti a partita IVA, possono vedere solo i propri clienti e contratti.
- Amministrazione contabile, può modificare i dati contabili dei clienti. Questo ruolo può altresì esportare le voci contrattuali all'**ERP!** per gestire la fiscalità.

- Project Manager con compito di revisionare tecnicamente il contratto. Questo compito è volto alla gestione a monte dell'invenduto.
- Visualizzatore senza permessi di scrittura.

Gestione dei contratti

Nella creazione dei contratti, core del progetto, si dovrà sviluppare il vero salto di qualità. Nelle interviste si è cercato di raccogliere le richieste che, anche dai primi rilasci, potessero dare al richiedente un sistema più ottimale. Tali richieste sono:

- Gestione storico dei dati in caso di modifica.
- Gestione di più versioni di uno stesso contratto legati dal medesimo codice progressivo. In caso di approvazione serve far decadere le versioni non approvate.
- Importazione dal **PLM!** di dati tecnici dei prodotti. Questi dati sono fondamentali nei calcoli utili alla contrattazione (Es: Potenza elettrica totale consumata).
- Gestione degli stati di avanzamento di un progetto e relative notifiche.
- Revisione tecnica del venduto in modo da dare la possibilità di proporre all'occorrenza integrazioni contrattuali.

Capitolo 4

Analisi dei requisiti

4.1 Requisiti funzionali

Questa sezione è dedicata alla suddivisione dei dati raccolti in requisiti funzionali. La tabella, nella figura ?? seguente, suddivide i requisiti per area del progetto. Insieme al requisito e alla sua descrizione è presente anche l'elenco dei ruoli a cui esso è rivolto.

4.1. REQUISITI FUNZIONALI

Area / Modulo	Requisito Funzionale	Descrizione / Specifiche	Ruoli Coinvolti
Sicurezza & Accessi	Profilazione RBAC granulare	Gestione permessi per tabella (lettura/scrittura) tramite tabelle utenti e utenti_tipi.	Tutti i ruoli
Anagrafica Clienti	Controllo anti-duplicazione	Validazione obbligatoria dell'univocità tramite Partita IVA in fase di inserimento.	Amministrazione, Agenti, Commerciali
Anagrafica Clienti	Sedi multiple e tipizzazione	Gestione sedi legali/operative, destinazioni merce separate e distinzione cliente finale / rivenditore.	Amministrazione, Agenti
Anagrafica Clienti	Storico contratti cliente	Consultazione integrata di tutte le offerte e ordini emessi per lo specifico cliente.	Commerciali, Amministrazione
Catalogo Prodotti	Codifica univoca	Allineamento del codice prodotto tra Configuratore, Ad-Hoc ERP e Fusion PLM.	Admin, Commerciali
Catalogo Prodotti	Gestione obsoleti e visibilità	Blocco dei componenti obsoleti rilevati da PLM; flag di visibilità per migrazione progressiva da ERP.	Admin, PM / Engineering
Catalogo Prodotti	Selezione per Macrogruppi	Navigazione guidata ad albero (Macchina → Componenti → Accessori → Ricambi) con vincoli di compatibilità.	Commerciali Nuovi/Ricambi
Catalogo Prodotti	Scheda caratteristiche tecniche	Attributi tabellari strutturati: meccanici, elettrici, pneumatici, logistici e accessori correlati.	Commerciali, PM / Engineering
Catalogo Prodotti	Descrizioni standardizzate	Modello ibrido: voce principale da menu standardizzato (da Fusion) affiancata da campo note libero.	Commerciali, PM / Engineering
Offerte & Pricing	Struttura a Capitoli	Suddivisione delle offerte in capitoli gestiti come sottodistinte personalizzabili.	Commerciali
Offerte & Pricing	Pricing centralizzato e override	Prezzi sincronizzati da Ad-Hoc con possibilità di inserimento manuale per voci speciali o su misura.	Commerciali, Admin
Offerte & Pricing	Scontistica multi-livello	Calcolo sconti combinabili: per riga prodotto, per capitolo, globale (esclusa MDO), globale (inclusa MDO).	Commerciali
Offerte & Pricing	Calcoli tecnici ed economici	Calcolo automatico di potenza elettrica, consumo aria compressa, pesi, volumi e stima costi montaggio/trasporto.	Commerciali
Offerte & Pricing	Vincoli Incoterms e Paesi	Selezione assistita Incoterms (con vincoli UE/extra-UE); adattamento automatico a valuta, lingua e normative (es. UL-CSA).	Commerciali, Logistica
Offerte & Pricing	Alert informativi	Notifiche a video non bloccanti per segnalare scelte non conformi alle specifiche standard d'impianto.	Commerciali
Offerte & Pricing	Versioning e clonazione	Storico revisioni tracciato (Rev.01, Rev.02...) e funzione di duplicazione rapida di offerte esistenti.	Commerciali
Ordini & Workflow	Conversione e congelamento	Blocco del contratto commerciale alla conferma cliente e assegnazione codifica causale ordine.	Commerciali, Amministrazione
Ordini & Workflow	Notifiche multi-reparto	Alert push automatici inviati a Amministrazione, PM, Logistica e Produzione all'apertura dell'ordine.	Sistema (Cross-funzionale)
Ordini & Workflow	Esportazione verso ERP	Invio automatizzato dell'ordine confermato ad Ad-Hoc per avviare la fatturazione.	Amministrazione
Revisione Tecnica	Contratto tecnico parallelo	Gestione di una variante tecnica interna (senza valore legale) gestita dal PM con costificazione separata.	Project Management
Revisione Tecnica	Workflow decisionale modifiche	Bivio post-revisione: approvazione commerciale con nuovo contratto/RDA (impatto alto) o gestione interna (impatto basso).	PM, Commerciale
Stampe & Documenti	Layout di stampa per reparto	Generazione PDF personalizzati per Cliente, Amministrazione e Ufficio Tecnico.	Tutti i reparti
Localizzazione	Supporto multilingua	Interfaccia localizzata e traduzione automatica/manuale delle descrizioni contrattuali.	Commerciali, Agenti

Figura 4.1: Requisiti funzionali raccolti.

4.2 Requisiti non funzionali

Questa sezione, a differenza della precedente raccoglie i requisiti non funzionali. La tabella che seguirà, nella figura ??, insieme al requisito stesso, raccoglie anche la sua categoria e la sua priorità all'interno dell'implementazione. I requisiti sono ordinati per priorità.

Categoria	Requisito Non Funzionale	Priorità	Specifica Tecnica / Obiettivo
Affidabilità	Eliminazione crash e blocchi	Critica	Stabilità operativa durante inserimento massivo righe, calcoli e stampa, azzerando i blocchi tipici di Access.
Interoperabilità	Connessione nativa ERP	Critica	Integrazione bidirezionale diretta via query SQL Server con Ad-Hoc (anagrafiche, listini ed export ordini).
Disponibilità	Architettura offline (LocalDB)	Alta	Continuità operativa garantita anche in assenza di rete o VPN tramite istanza locale SQL Server LocalDB.
Integrità Dati	Sincronizzazione automatica	Alta	Motore di sync bidirezionale tra LocalDB e SQL Server aziendale con logica di risoluzione conflitti.
Resilienza	Backup automatici in VPN	Alta	Salvataggio automatico del database locale durante le connessioni VPN per prevenire perdite o corruzioni dati.
Integrità Dati	Unificazione codici prodotto	Alta	Coerenza assoluta delle codifiche articolo tra Configuratore, Ad-Hoc ERP e Fusion PLM senza disallineamenti.
Usabilità	Workflow guidati (Wizard)	Alta	Percorsi passo-passo che guidano la compilazione per azzerare omissioni o errori di configurazione commerciale.
Usabilità	Viste orientate al ruolo (RBAC)	Alta	Maschere e sezioni della UI mostrate dinamicamente in base alle prerogative del ruolo utente loggato.
Portabilità	Framework moderno	Media	Architettura per ambiente desktop con predisposizione a target mobile.
Prestazioni	Calcolo istantaneo in-memory	Media	Elaborazione immediata a runtime di formule per potenze, consumi d'aria, pesi, volumi e stime logistiche.
Qualità Processo	Area di staging ordini	Media	Area di transito temporanea per la revisione amministrativa delle conferme d'ordine prima del caricamento contabile.
Evolutività	Predisposizione CAD / PLM	Bassa	Architettura aperta al recupero di disegni tecnici e visualizzazione di modelli 3D direttamente da Fusion.

Figura 4.2: Requisiti non funzionali raccolti.

Capitolo 5

Progettazione dell'architettura

5.1 Preambolo sulle scelte architetturali

Le scelte architetturali sono dipese in primo luogo dall'infrastruttura aziendale, e successivamente dalle richieste durante gli incontri. L'utilizzo di SQLServer, come motore di database, è stato preso come base di tutte le scelte successive. Da questo caposaldo, si è proseguito con la ricerca di un framework che interagissero sia con SQLServer che SQLite.[?]. La necessità di operare offline ha guidato la ricerca di un sistema installabile in locale, escludendo così le soluzioni web based. La scelta è quindi ricaduta sulla piattaforma .NET[?] con linguaggio C# che, in quanto dell'ecosistema Microsoft, è perfettamente integrata con i database prima citati. La piattaforma .NET è espandibile mediante l'utilizzo di NuGet[?], che permette di aggiungere package e quindi funzionalità al progetto. I pacchetti sono sviluppati sia da Microsoft che da terze parti, e vengono aggiornati e supportati continuamente. Questa manutenzione continua evita di incorrere in debiti tecnici futuri. L'ambiente .NET è nativamente multiplatforma, questa proprietà soddisfa le necessità dei richiedenti. Secondo le linee guida Microsoft, per lo sviluppo delle view multiplatforma, è consigliato l'utilizzo del package NuGet .NET MAUI[?]. In fase di compilazione, MAUI converte il codice sorgente in codice nativo per la piattaforma di destinazione. MAUI integrato con il package NuGet "Community Toolkit Maui"[?] consente la possibilità di costruire le interfacce grafiche in codice C#. Nativamente MAUI supporta solo il linguaggio Markup XAML[?].

5.2 Deployment

La scelta degli ambienti di sviluppo prima citati, porta a dover considerare il deployment dell'applicativo. Il software verrà installato in locale su ogni dispositivo, sia esso desktop o mobile, e si conatterà al database centrale. All'interno dell'azienda è presente un dominio **AD!** (**AD!**), dove ogni Oggetto del dominio ha un **IP!** (**IP!**) statico. Con questa configurazione, la connessione al database, risiedente in un server all'interno del dominio, è facilmente raggiungibile tramite **IP!** se si è all'interno della **LAN!** (**LAN!**). Lo scambio di dati tra il frontend e il backend avviene mediante protocollo **TCP/IP!** (**TCP/IP!**). Si è valutato, per chi accede dall'esterno del dominio, di utilizzare la rete **VPN!**, stabilendo una connessione virtuale che permette di essere all'interno della **LAN!**. Il database locale, in SQLite, è strutturato come una copia del database centrale. I dati vengono sincronizzati tra i due database mediante un processo di sincronizzazione. In assenza di connessione, verificabile tramite ping, si accede al database locale, e ristabilita la connessione, i dati vengono sincronizzati. Il diagramma dei deployment è riassunto in figura ??.

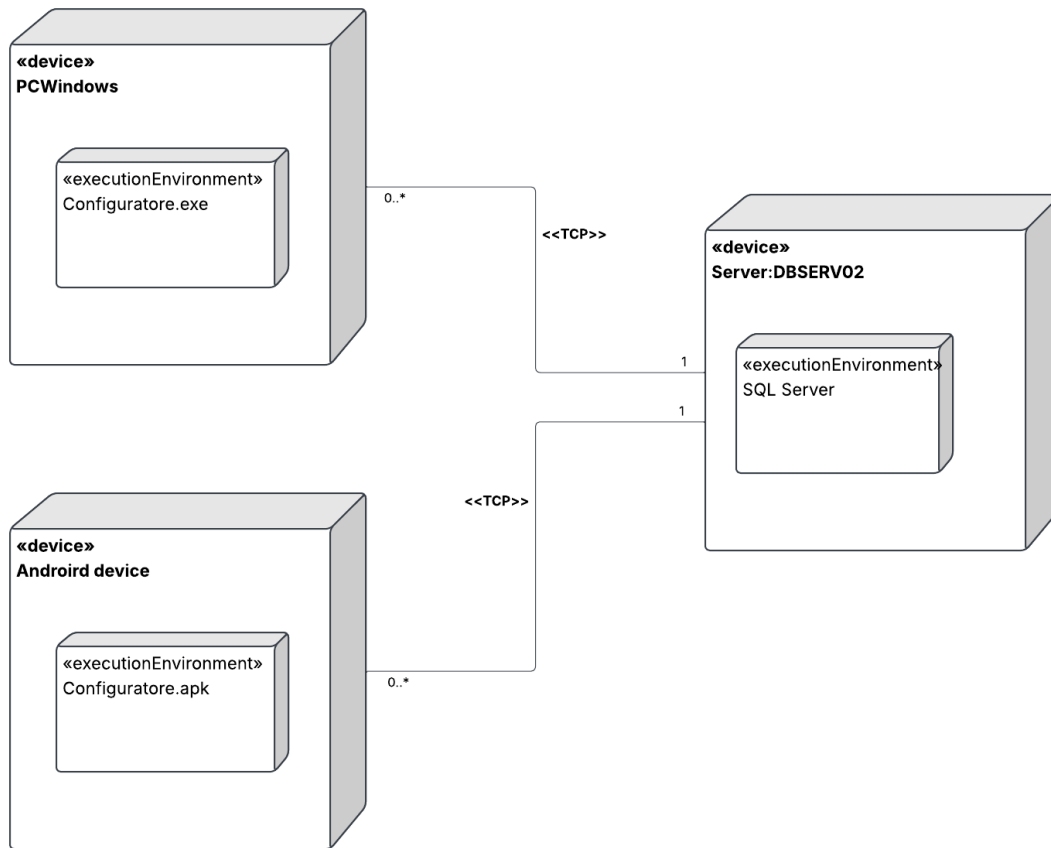


Figura 5.1: Diagramma dei deployment

Nella fase di sviluppo, per testare la struttura del database, è stato scelto di utilizzare una versione di SQLServer locale, denominata SQL Express[?]. SQL Express condivide con SQL Server tutte le caratteristiche principali, questa proprietà ha permesso di costruire query e testare funzionalità localmente, ma compatibili con il database di produzione.

5.3 Package

La progettazione dell'applicativo ha portato a suddividerlo in tre Package con compiti ben distinti.

5.3.1 Interfaccia al database

Il package di interfaccia al database, denominato “DatabaseManagementPackage”, ha il compito di gestire tutti gli aspetti legati a quest’ultimo, come:

- Gestire connessioni vari database, come SQLite e SQL Server o, nel caso dei test SQL Express.
- Gestione delle transazioni.
- Gestione dei permessi utente, bloccando le operazioni non consentite.
- Conversione, mediante **ORM! (ORM!)**, delle tabelle database in entity.
- Mappare entity in file neutri, proteggendo il database.
- Gestione di Repository per ogni entità, che espongono le operazioni CRUD.@
- Gestione di una suite di test per verificare le operazioni precedenti. Questa suite è risultata fondamentale nell’evoluzione del database, nel corso dell’implementazione ad ogni cambiamento si sono validati i risultati.
- Esporre al di fuori del package solo file neutri e classe di manager del database.

Nella implementazione si è utilizzato il pacchetto NuGet **EF Core! (EF Core!)[?]**. Questo modulo permette una gestione efficiente delle operazioni di accesso ai dati, consentendo di astrarre le query SQL in codice **C#**. La mappatura delle entity in **DTO! (DTO!)**, proteggendo i dati, è stata realizzata mediante il package “Mapster”[?], appartenente a terze parti. La mappatura entity-**DTO!** è stata realizzata in modo bidirezionale, in quanto in alcuni casi è necessario convertire un **DTO!** in entity per poterlo salvare nel database.

5.3.2 Libreria di componenti View

La creazione di interfacce grafiche è stata supportata dalla creazione di un package dedicato, che racchiude in sé factory di componenti, costanti e utilità. Il package, denominato “ViewComponentLibrary”, è stato concepito a sé stante per favorire

un suo impiego in implementazioni future, in quanto racchiude componenti generici e riutilizzabili. La libreria si appoggia a sua volta al package NuGet “Community Toolkit Maui” [?]. Questo package permette di scrivere le interfacce grafiche in codice C# Markup, evitando quindi l’utilizzo del linguaggio XAML[?], standard di sviluppo per le interfacce MAUI.

5.3.3 Modulo Main e View

Il package principale, ed il main del progetto, è quello dedicato all’orchestrazione dei componenti e alla gestione dei flussi di lavoro. Il package, denominato “ConfiguratoreCommerciale”, ha numerosi compiti, tra cui la gestione della Dependency Injection, la creazione delle view e dei rispettivi ViewModel. In questo package risiedono infatti i ViewModel, che si interpongono tra le view e i repository, esponendo i dati tramite “Observable Properties”. Le Dependency Injection invece permettono di iniettare le dipendenze necessarie alle classi, come ad esempio i repository, classi di Logging o le view stesse se registrate. Le classi di view, prendendo i componenti dalla libreria prima costruita, vengono realizzate qui all’interno differenziando l’uso di un componente o un altro in base al dispositivo. Questo aspetto è permesso dall’utilizzo di condizioni di compilazione, che permettono di distinguere il codice da compilare in base alla piattaforma di destinazione. Un esempio di tale condizione è il seguente:

```
1 #if ANDROID
2     // codice per piattaforma Android
3 #elseif WINDOWS
4     // codice per piattaforma Windows
5 #endif
```

Questa possibilità permette, inoltre di eseguire parti di codice in solo in fase di DEBUG. Gli aspetti generali gestiti possono essere riassunti in un elenco puntato, come segue:

- Creazione interfacce grafiche.
- Traduzione a compile time della conversione di codice C# in XAML.
- Traduzione a compile time in elementi nativi per la piattaforma di destinazione.

- Gestione delle interfacce mediante ViewModel, che a loro volta tramite observable modificano la view in risposta ad eventi.
- Gestione della navigazione tra le interfacce grafiche.
- Gestione dei permessi, mostrando solo i comandi che competono al ruolo dell'utente.
- Mediante la classe MauiProgram.cs gestione della Dependency Injection.
- In fase di avvio creazione di una cache in RAM di dati immutabili durante la sessione, come tabelle di lookup.

Il diagramma dei package è riassunto in figura ??.

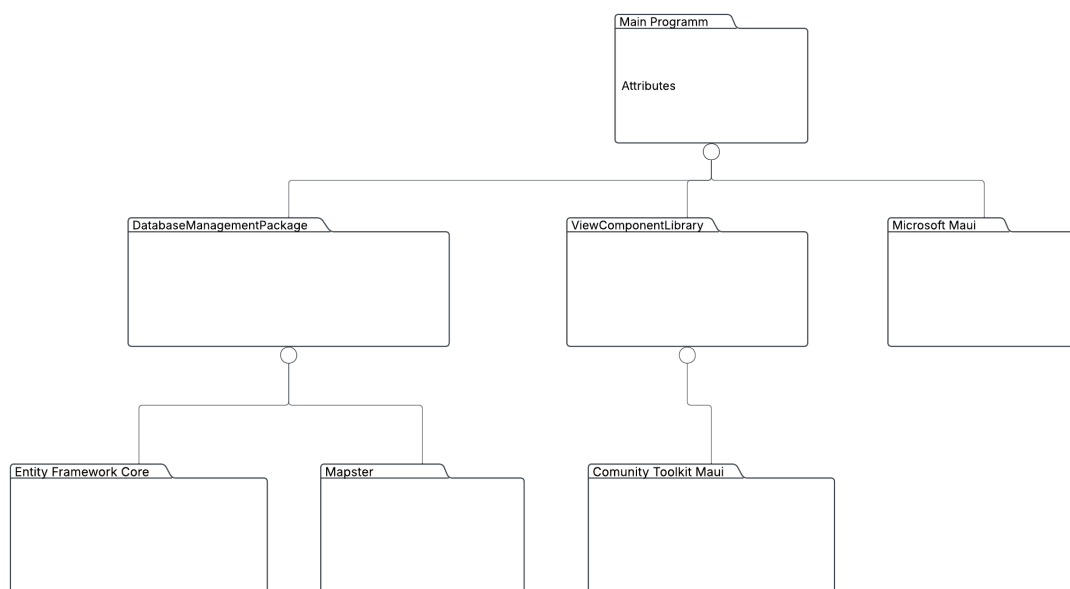


Figura 5.2: Diagramma dei package

5.4 Flussi di lavoro

Osservando i requisiti si sono osservate delle problematiche relative all'attuale flusso aziendale del contratto. Si è quindi studiato di inserire una revisione tecnica,

Capitolo 6

Scelte progettuali

6.1 Scelta modello di sviluppo

La scelta del modello progettuale è dipesa dalla necessità di prototipare e testare le funzioni nell'arco dell'intero sviluppo con gli utenti. Si è quindi deciso di adottare un modello agile, supportato dalla creazione e test dei prototipi di funzioni o interfacce. La progettazione, come da canone agile, è stata quindi iterativa e incrementale. Tutte le nuove funzioni implementate sul database sono state testate da apposite Unit Test che ne validassero il comportamento. Tali test di regressione sono stati fondamentali per validare le modifiche al database, in quanto esso è stato soggetto a continue modifiche di perfezionamento. Le interfacce grafiche, al contrario, non sono state supportate da test, ma prototipate e validate da utenti scelti. La rifattorizzazione del codice è stata affrontata con regolarità seguendo il principio **DRY!** (**DRY!**). Nel seguire il principio, si sono create più factory e classi di utilità che unificassero quanto più possibile il codice comune.

6.2 Suddivisione e contenuto dei rilasci

Le tempistiche dei rilasci sono state decise insieme ad un Project Management aziendale che mi ha supportato nella stima dei tempi di sviluppo e nella conversione del database esistente nel nuovo. Nel primo rilascio, si è scelto di includere tutti i

requisiti già presenti nel precedente applicativo. Tale rilascio, con consegna stimata gennaio 2027, sono presenti le seguenti funzionalità:

- Gestione utenti e permessi.
- Anagrafica clienti.
- Creazione delle offerte.
- Versioning offerte.
- Creazione ordini.
- Import ed export dati, da e verso altri database.
- Creazione reportistica.
- Gestione multilingua interfaccia applicativa.
- Prototipo mobile.
- Modulo di configurazione prodotto.

Si è scelto di non includere la funzionalità offline, in quanto essa necessita di un database centrale stabile. La funzionalità offline verrà implementata nel secondo rilascio. Questa fase avverrà una volta che il software sarà in produzione, per poter contare con un database pressochè stabile. Nel secondo rilascio saranno presenti quindi:

- Gestione revisione tecnica e differenze con contratto firmato.
- Inserimento calcoli automatici sulla base delle caratteristiche del venduto (ES:potenza elettrica totale assorbita).
- Gestione multi-valuta per ordini internazionali.
- Apertura disegni PDF dal server del **PLM!**.
- Report avanzati.
- Implementazione database locale con logiche di sincronizzazione.

6.3 Scelta Framework ORM

All'interno della sezione sull'architettura ho citato la scelta del framework **EF Core!??**. Questo framework ha permesso di mappare le tabelle mediante la tecnica **ORM!**. Questa tecnica consiste nel mappare ogni tabella del database in una classe che ne rappresenta l'entità. I legami tra le entità sono gestiti mediante la classe context. **ORM!** ha permesso, mediante script powershell e connessione al database, di ricreare in modo rapido tutte le entity, ed il context, a fronte di cambiamenti del database. Tale approccio può essere definito database first, in quanto la progettazione avviene principalmente sul database, ed il software nel ereditare le modifiche. Le query sono state scritte tramite le funzioni **LINQ!** (**LINQ!**) che hanno permesso di scriverle in linguaggio **C#**.

Listing 6.1: Esempio di query LINQ

```
1 public class Example{
2     public async Task<ClientiDestinazioniDTO> GetLocationDetailsById(int?
        idLocation)
3     {
4         ClientiDestinazioni? location = await _db.ClientiDestinazioni
            .Where(l => l.CdeId == idLocation)
5             .Include(l => l.CdeNazioneNavigation)
6             .Include(l => l.CdeTipoNavigation)
7             .Include(l => l.CdeClienteNavigation)
8             .Include(l => l.CdeNormaElettricaNavigation)
9             .Include(l => l.CdeZonaNavigation)
10            .AsNoTracking()
11            .FirstOrDefaultAsync();
12        return _mapper.Map<ClientiDestinazioniDTO>(location!);
13    }
14 }
15 }
```

In ottica di Offline le query in **LINQ!**, sono compatibili con il database locale SQLite se si limitano le funzioni utilizzate a quelle supportate da esso. Le funzioni compatibili sono ad esempio: *Where*, *Select*, *OrderBy*, *GroupBy*, *Join*, *Distinct*.

All'interno degli aspetti negativi era presente la lentezza in grandi volumi dovuti alla traduzione del codice in query. Si è valutato, in caso si presentino di ottimizzare il database mediante indici in prima battuta e successivamente ottimizzando le singole query.

6.4 Scelta Framework grafico

Il framework grafico scelto doveva soddisfare il requisito della portabilità e dell'offline. Le scelte vagliate sono state:

- Front end web based, con utilizzo di html, css, javascript e mediante api interrogare il database centrale.
- Utilizzo del Framework MAUI, nativamente cross-platform ed integrato perfettamente con Backend.

L'interfaccia web, utile per avere view che si adattano a dispositivi diversi, presentavano delle criticità. La difficoltà principale risiedeva nell'utilizzo in locale. In assenza di connessione tutti i file ed il database locale devono essere presenti sul dispositivo stesso. Questa caratteristica rende difficoltoso gestire gli aggiornamenti. Il secondo motivo è l'utilizzo di più linguaggi di programmazione che non massimizzano l'esperienza acquisita con altri moduli. In ottica di ridurre la tempistica nell'avere un primo test con gli utenti essa era considerata un malus. Il framework MAUI di .NETsopperisce ad entrambi i problemi. Il programma è fisicamente installato sui dispositivi ed il loro aggiornamneto può essere gestito centralmente all'interno di uno store Microsoft. Essendo esso in base .NETe, mediante il framework "Comunity Toolkit Maui", le interfacce sono scritti in base C#.

6.5 Scelta del pattern di progettazione

Insieme all'utilizzo di MAUI è suggerito da Microsoft il pattern **MVVM!** (**MVVM!**). Il modello MVVM CONSENTE DI SEPARARE IN MODO PULITO LA LOGICA DI BUSINESS E PRESENTAZIONE DI UN'APPLICAZIONE DALLA RELATIVA INTERFACCIA UTENTE. LA GESTIONE DI UNA SEPARAZIONE NETTA TRA LA LOGICA DELL'APPLICAZIONE E L'INTERFACCIA UTENTE CONSENTE DI RISOLVERE NUMEROSI PROBLEMI DI SVILUPPO E SEMPLIFICA IL TEST, LA GESTIONE E L'EVOLUZIONE DI UN'APPLICAZIONE. PUÒ ANCHE MIGLIORARE SIGNIFICATIVAMENTE LE OPPORTUNITÀ DI RIUTILIZZO DEL CODICE E CONSENTE AGLI SVILUPPATORI E AI PROGETTISTI DELL'INTERFACCIA UTENTE DI COLLABORARE PIÙ FACILMENTE DURANTE LO SVILUPPO DELLE RISPETTIVE PARTI DI UN'APP.[?] IL MODELLO

MVVM! È COMPOSTO DA TRE ELEMENTI FONAMENTALI CHE INTERAGISCONO COME IN FIGURA ??:

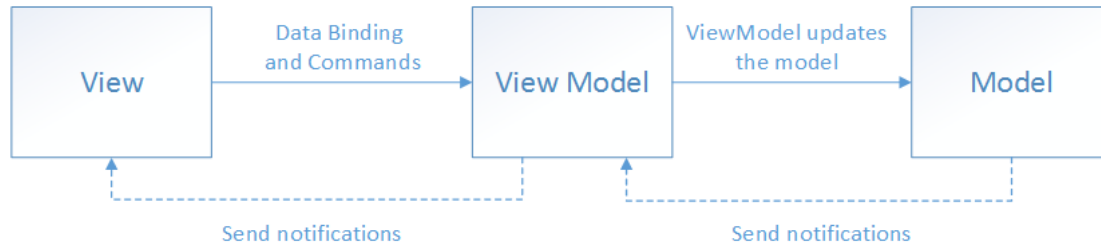


Figura 6.1: Diagramma funzionamento MVVM

QUESTA MODELLO, SEPARANDO LA LOGICA DELLA VIEW DA QUELLA DI BUSINESS, HA PERMESSO DI PROTOTIPARE LE INTERFACCE NEI PRIMI RILASCI E IN FUTURO RIPROGETTARLE MIGLIORANDO L'ACCESSIBILITÀ E UX! (UX!).

Capitolo 7

Progettazione del database

7.1 Esportazione Vecchio Db

7.2 Denominazione tabelle e campi

7.3 Utilizzo di SSIS nel processo di migrazione

7.4 Ristrutturazione database

7.4.1 Eliminazione ridondanze e normalizzazione

7.4.2 Creazione tabelle di lookup

7.4.3 Aggiunta tabelle non presenti

7.4.4 Creazione tabelle di Back Up

7.4.5 Scelta di strutture Avanzate come Triggers e Stored Procedure

7.5 Schema logico finale

Capitolo 8

Creazione Model

8.1 Mapping delle classi

8.2 Struttura del package

8.2.1 Classi di Test

8.2.2 Repository

8.2.3 Classi ed Interfacce esposte

Capitolo 9

Creazione View

9.1 Utilizzo di Figma per disegno interfacce

9.2 Creazione delle Factory per le View

9.3 Scelta delle classi di ViewModel

Capitolo 10

Modulo configurazione prodotto

10.1 Scopo del modulo

10.2 Algoritmo di configurazione

10.3 Realizzazione sul database

10.4 Interfaccia grafica lato utente ed amministratore

Bibliografia

- [FIG] FIGMA. FIGMA. [HTTPS://WWW.FIGMA.COM/](https://www.figma.com/).
- [MAU] .NET MAUI. .NET MAUI. [HTTPS://LEARN.MICROSOFT.COM/IT-IT/DOTNET/MAUI/WHAT-IS-MAUI?VIEW=NET-MAUI-10.0](https://learn.microsoft.com/it-it/dotnet/maui/what-is-maui?view=net-maui-10.0).
- [MICA] MICROSOFT. MAUI.MVVM. [HTTPS://LEARN.MICROSOFT.COM/IT-IT/DOTNET/ARCHITECTURE/MAUI/MVVM](https://learn.microsoft.com/it-it/dotnet/architecture/maui/mvvm).
- [MicB] MICROSOFT. MICROSOFT ENTITY FRAMEWORK. [HTTPS://LEARN.MICROSOFT.COM/EN-US/EF/](https://learn.microsoft.com/en-us/ef/).
- [MicC] MICROSOFT. SQL SERVER. [HTTPS://WWW.MICROSOFT.COM/SQL-SERVER](https://www.microsoft.com/sql-server).
- [MicD] MICROSOFT. SQL SERVER EXPRESS. [HTTPS://LEARN.MICROSOFT.COM/EN-US/SQL/DATABASE-ENGINE/CONFIGURE-WINDOWS/SQL-SERVER-EXPRESS-LOCALDB?VIEW=SQL-SERVER-VER17](https://learn.microsoft.com/en-us/sql/database-engine/configure-windows/sql-server-express-localdb?view=sql-server-ver17).
- [MICE] MICROSOFT. XAML LANGUAGE. [HTTPS://LEARN.MICROSOFT.COM/EN-US/DOTNET/DESKTOP/WPF/XAML/](https://learn.microsoft.com/en-us/dotnet/desktop/wpf/xaml/).
- [.NE] .NET. .NET. [HTTPS://DOTNET.MICROSOFT.COM/](https://dotnet.microsoft.com/).
- [NuG] NUGET. NUGET PACKAGE MANAGER. [HTTPS://LEARN.MICROSOFT.COM/IT-IT/NUGET/](https://learn.microsoft.com/it-it/nuget/).
- [OPEA] OPENSOURCE. COMMUNITYTOOLKIT.MAUI. [HTTPS://LEARN.MICROSOFT.COM/IT-IT/DOTNET/COMMUNITYTOOLKIT/MAUI/](https://learn.microsoft.com/it-it/dotnet/communitytoolkit/maui/).
- [OPEB] OPENSOURCE. MAPSTER. [HTTPS://GITHUB.COM/MAPSTERMAPPER/MAPSTER](https://github.com/MapsterMapper/Mapster).

BIBLIOGRAFIA

- [RUL] RULE DESIGNER. FUSION. [HTTPS://WWW.RULEDESIGNER.COM/IT/](https://www.ruledesigner.com/it/).
- [SQL] SQLITE. SQLITE. LIGHTWEIGHT DATABASE ENGINE, [HTTPS://WWW.SQLITE.ORG/](https://www.sqlite.org/).
- [ZUC] ZUCCHETTI. AD-HOC. [HTTPS://WWW.ZUCCHETTI.COM/IT-IT/SOLUZIONI/GESTIONALI](https://www.zucchetti.com/it-it/soluzioni/gestionali).

Acknowledgements

OPTIONAL. MAX 1 PAGE.